

Hyun
Rundel

SPARK

Notes



Big Data → Grande Volume / Grande Varietà / Grande Velocità
3V

MapReduce → modello di programmazione dei parametri di processo parallelamente ed in modo distribuito un enorme quantità di dati.

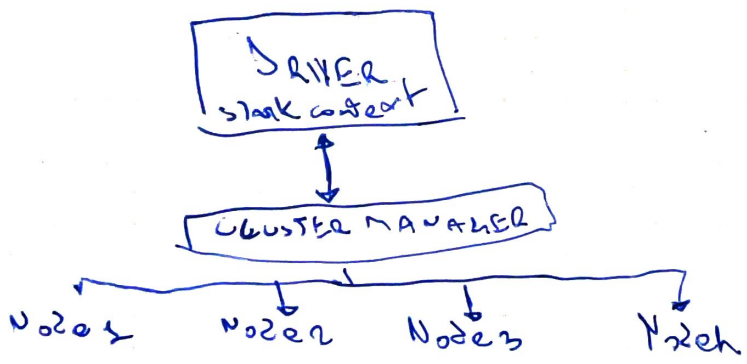
Spark → open source distribuito → Memory cache
Minimizza le query.
↓
Eccellenza nella memoria

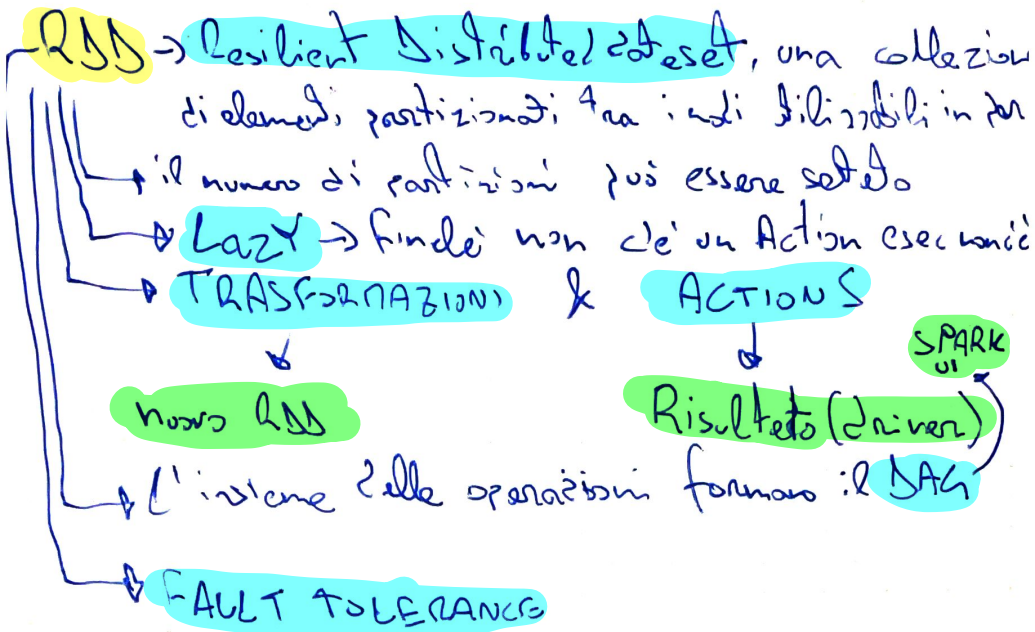
Python, Scala, Java, R

Batch, real time, ML, Graph

Moduli: MLlib, Streaming, SQL, GraphX

Un'applicazione spark funziona indipendentemente su un cluster coordinato dal SPARK CONTEXT





TRASFORMAZIONI

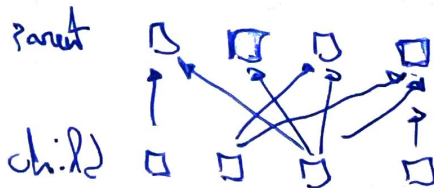
NARROW

Ogni partizione nel RDD figlio sono semplici dipendenze finite e dirette. Possono essere eseguite in modo arbitrario senza necessità di info sugli altri RDD.



WIDE

Bisogna eseguirli in accordo a qualcosa (es key). Induzione tutte le trasformazioni che chiamano "repartition".



TRASFORMAZIONI GRUND

MAP $\rightarrow$ Nuovo RDD con mapping

FILTER $\rightarrow$ Nuovo RDD con element. che rispettano cond.

FLATMAP $\rightarrow$ Uguale a MAP, ma ogni input $\rightarrow$ N output

MAPPARTITIONS $\rightarrow$ Map da runna sulle diverse partizioni

MAPPARTITIONSWITHINDEX $\rightarrow$ Same + indice partizione

SAMPLE $\rightarrow$ Prende un sample dall RDD di input

UNION $\rightarrow$ element. nel RDD più quelli nell'arg

INTERSECTION $\rightarrow$ Size in 2 input di arg

DISTINCT $\rightarrow$ Distinct dell RDD

GROUPBYKEY $\rightarrow (K, V) \rightarrow (K, \text{Iterable}[V])$

REDUCEBYKEY $\rightarrow$ Per ogni K ritorna un solo valore

AHHREDHATEBYKEY $\rightarrow$ Per ogni K ritorna un solo valore ^{univ. o più}

SORTBYKEYS $\rightarrow (K, V) \rightarrow (K, V)$ ordinato by chiave

JOIN $\rightarrow$ classico SQL ~~$(K, V) \rightarrow (K, V)$~~ join $(K, V) \rightarrow (K, (V, V))$

COGROUP $\rightarrow (K, V)$ e $(K, W) \rightarrow (K, \text{Iterable}[V], \text{Iterable}[W])$

CARTESIAN $\rightarrow$ prodotto cartesiano tra 2 RDD

PIPE $\rightarrow$ esegue comando real o bash su RDD (output stringa)

COALESCE $\rightarrow$ Diminuisce numero di partizioni

REPARTITION $\rightarrow$ Bilancia le partizioni, crea nuove

REPARTITIONSWITHSORTWITHINPARTITIONS $\rightarrow$ v.g. a data

Azioni Comuni

- REDUCE → Aggrega elementi usando una funzione
- COLLECT → Ritornare tutto il dataset al driver sotto forma Array
- COUNT → Numero degli elementi
- FIRST → primo elemento
- TAKE → primi N elementi di un RDD
- TAKESAMPLE → Array di sample del dataset
- TAKESAMPLEWITHREPLACEMENT → primi N elem ordinati, con rimpiazzamento
- SAVEASTEXTFILE → quello che dice
- SAVEASSEQUENCEFILE → ^{solo k,v} scrivere sotto forma di Mapas sequenziale
- SAVEASOBJECTFILE → Scrivere sotto forma di JAVA SERIALIZE
- COUNTBYKEY → per ogni chiave contare
- FOREACH → per ogni elemento, chiama una funzione

SHUFFLE

Avviene quando bisogna redistribuire i dati per raggrupparli intorno alle partizioni.
Tipicamente includono copie di dati attraverso gli **executor** e rappresentano un'operazione **costosa e complessa**.

Questi dati vengono distribuiti tra gli **executor** sotto forma di partizioni. Ognuna di queste viene gestita da un **executor**.

Spesso i dati vengono divisi in partizioni in base ad una **key**, se questa non è bilanciata si crea un sovraccarico su una partizione riducono i tempi di esecuzione di tutto il programma.

Lo **shuffle** interviene quando i dati vengono **riorganizzati** tra le partizioni, spesso quando una trasformazione necessita di info da altre partizioni. Spark si occupa di **raccolgere i dati ridistribuiti da ciascuna partizione e combinarli in una nuova**

Sort più lento ed usa **più memoria** rispetto

Map, che è quello di default

Si lavora con: **Spillin reduction, Compression, Serializ.** Sick...

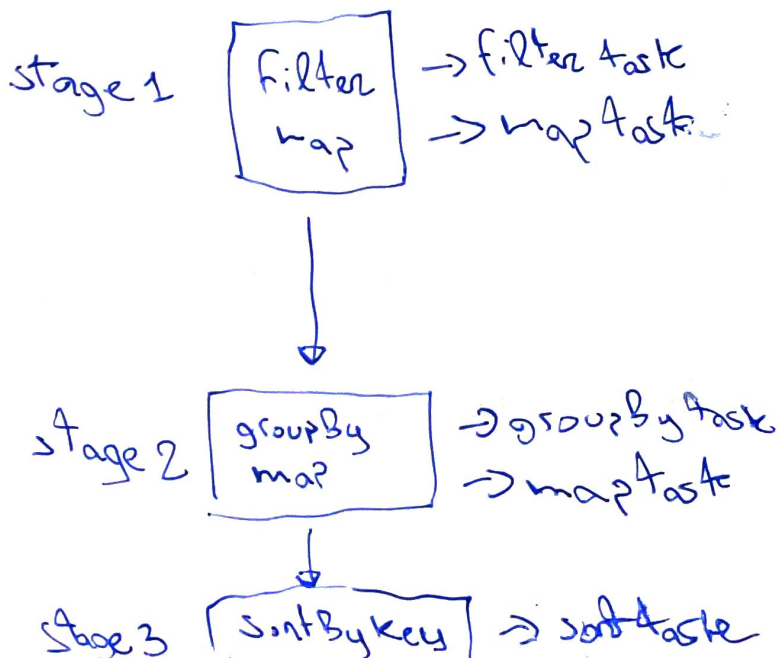
Scheduling

DAG → 11 stages per ogni Spark Job

Job → corrisponde ad una azione, ed ogni azione è chiamata dal DRIVER di un applicazione SPARK.

STAGE → Ogni stage corrisponde ad una dipendenza di shuffle o ad una WRIT transformation. Uno stage è l'insieme delle operazioni che un esecutore può effettuare senza comunicare all'esterno.

TASK → Gruppo di uno stage. E' l'insieme delle operazioni effettuate su una singola partizione/singolo esecutore.



SHARED VARIABLE

Broadcast variable permettono di creare una variabile READ-ONLY da condividere a tutti gli esecutori

ACCUMULATOR sono variabili che permettono solo aggiunte, adatti quindi per somme e riduzioni, sono numerici ma si possono definire nuovi.

SPARK SQL

Con l'arrivo di Spark SQL Spark introduce un nuovo motore dati per velocizzare le esecuzioni: DFE

1 Dataframe permission di fare tutte le possibili operazioni normalmente fattibili in SQL

Si possono creare tabelle ^{anche global per share} temporanee partendo da un df per poter fare query sopra.

Stream Processing

Structured Streaming → Based on Spark SQL

Spark Streaming → Based on Spark Core

Problems on Streaming

- Consistency record possono essere elaborati in una parte prima che in un'altra portando a risultati senza senso
- Fault Tolerance → Va gestito l'errore
- Out-of-order data → I dati possono arrivare in disordine

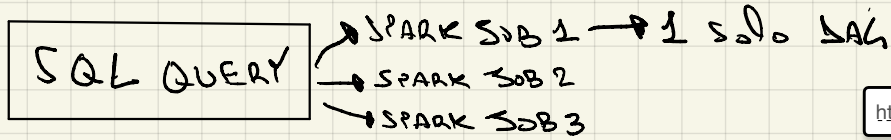
SPARK PLAN

E' possibile leggere il piano di esecuzione SPARK da **SPARK UI** o da **SPARK HISTORY SERVER**.

Quando l'esecuzione di un plan vengono fatti **CAST IMPLICIT**, e' sempre consigliato esplicitarli.

Le **Native READERWRITER** performano molto meglio delle **SERVS** (Velocità e stabilità).

Se una query è corrotta tutte le query seguenti saranno impattate.



<https://spark.apache.org/docs/3.0.0-preview/web-ui.html>

E' possibile visualizzare negli specifici **TAB**: **JOBS**, **STAGES**, **TASKS**. In essi si possono visualizzare info

Possibili problemi di performance per il piano di SPARK possono essere:

- Tabelle con migliaia di partizioni (NIVE metadata overhead)
- 100s di migliaia di milioni di file (FILE SYSTEM overhead)
- Nuovi dati non sono visibili, bisogna usare **REFRESH**

Delta Lake permette di risolvere tutti questi problemi.

Delta Lake + SPARK beneficia:

- Completamente ACID
- Schema Management
- Late Versioning e Time Traveling
- Batch/Streaming unificati
- Metadati scalabili
- Record aggiornati e cancellati
- Late Expectation

QUESTION

- Nel **TASK LIST** del **STAGE TAB** non è possibile vedere:
 - ☐ GC Time
 - ☐ Execution Time
 - ☐ Input Size
 - ☐ Shuffle Size
 - ☒ CPU Load
- You can read the plan from the [] in the [] tab. - **SPARK UI** - **SQL**
- Pushed Filter are visible in:
 - ☐ LOGICAL PLAN
 - ☒ PHYSICAL PLAN
 - ☐ OPTIMIZED PLAN

In **SPARK SQL** the physical plan that will be executed is:

- ☒ The best physical plan chosen by the planner
- ☐ The logical plan selected by optimizer
- ☐ The optimized plan in output from ^{optimizer}
- From which TAB is possible to execute a Thread Dump? **Execution TAB**
- A SQL query is corresponding to: **MULTIPLE SPARK JOB**
- The best way to understand where a stage is spending time is the [stage tab]. Where there is the Exec Timeline.

SPARK TUNING

Quando si crea un'applicazione SPARK bisogna pensare a quali configurazioni usare:

- `executor.memory` → essa + l'overhead deve essere minore alla dimensione del container
- `driver.memory` → più piccolo possibile senza che fallisca.
- `executor.vcores`
- `enable dynamic allocation`

Buoni suggerimenti sono:

- Ripartire le memorie del nodo tra i vari executor

- Preferire pochi grandi executor per nodo

- Se hai più piccoli executor ci sono più ritardi di comunicazione

- Decidere quanti usare in base al problema.

- 1 core = 1 task, il numero dei task è limitato agli executor, troppi però possono creare problemi. ^{thread safe}

- 1 solo core per executor fa perdere benefici quali BROADCAST

DYNAMIC ALLOCATION

Permette a SPARK di aggiungere e sottrarre executor tra i vari job dell'applicazione.

Si settano 4 variabili:

- To configure
 - `spark.dynamicAllocation.enabled=true`
 - `spark.shuffle.service.enabled=true` (you have to configure external shuffle service on each worker)
 - `spark.dynamicAllocation.minExecutors`
 - `spark.dynamicAllocation.maxExecutors`
 - `spark.dynamicAllocation.initialExecutors`
- To Adjust
 - Spark will add executors when there are pending tasks (`spark.dynamicAllocation.schedulerBacklogTimeout`)
 - and exponentially increase them as long as tasks in the backlog persist (`spark...sustainedSchedulerBacklogTimeout`)
 - Executors are decommissioned when they have been idle for `spark.dynamicAllocation.executorIdleTimeout`

Perché?

- When
- Most important for shared or cost sensitive environments
 - Good when an application contains several jobs of differing sizes
 - The only real way to adjust resources throughout an application
- Improvements
- If jobs are very short adjust the timeouts to be shorter
 - For jobs that you know are large start with a higher number of initial executors to avoid slow spin up
 - If you are sharing a cluster, setting max executors can prevent you from hogging it

Quando ci sono problemi di memoria si può provare:

- Se si ha più memoria provare a concederla

- Provare ad aumentare il numero delle partizioni (in una funzione o `spark.default.parallelism`)

- Se ci sono execution in idle probabilmente abbiamo sotto-occupati i nostri executor.

- Se tutti sono in idle c'è un'operazione onerosa nel driver.

- Se è un'operazione complessa ma esce in idle aumentare partizioni

Prevenire shuffle spill to disk: - large executors - Configure offheap space - più partizioni possono aiutare - Settare shuffle setting

Tuning Can Help With

- Low cluster utilization (\$\$\$\$)
- Out of memory errors
- Spill to Disk / Slow shuffles
- GC errors / High GC overhead
- Driver / Executor overhead exceptions
- Reliable deployment

Things you can't "tune away"

- Silly Shuffles
 - You can make each shuffle faster through tuning but you cannot change the fundamental size or number of shuffles without adjusting code
 - Think about how much you are moving data, and how to do it more efficiently
 - Be very careful with column based operations on Spark SQL
- Unbalanced shuffles (caused by unbalanced keys)
 - If one or two tasks are much larger than the others
 - #Partitions is bounded by #distinct keys
- Bad Object management / Data structures choices
 - Can lead to memory exceptions / memory overhead exceptions / gc errors
- Serialization "exceptions"
 - "Exceptions due to cost of serialization can be tuned"
- Python
 - This makes Holden sad, buuuut, Arrow?

Il tuning può dipendere da 4 fattori:

- 1- Execution Environment

- 2- Size dell'input

- 3- Tipo di computazione

- 4- History del run del job

Execution Environment

What we need to know about where the job will run

- How much memory do I have available to me: on a single node/ on the cluster
- In my queue
- How much CPU: on a single node, on the cluster → corresponds to total number of concurrent tasks
- We can get all this information from the yarn api (<https://dzone.com/articles/how-to-use-the-yarn-api-to-determine-resources-ask-1>)
- Can I configure dynamic allocation?
- Cluster Health

Size of the input data

- How big is the input data on Disk
- How big will it be in memory?
 - Coefficient of In Memory Expansion: shuffle Spill Memory / shuffle Spill disk
- Can get historically or guess
- How many part files? (the default number of partitions at read)
- Cardinality and type?

- La prima prova è quella di usare le partizioni

How much memory should each task use?

```
def availableTaskMemoryMB(executorMemory : Long): Double = {  
  val memFraction = sparkConf.getDouble("spark.memory.fraction", 0.6)  
  val storageFraction = sparkConf.getDouble("spark.memory.storageFraction", 0.5)  
  val nonStorage = 1 - storageFraction  
  val cores = sparkConf.getInt("spark.executor.cores", 1)  
  Math.ceil((executorMemory * memFraction * nonStorage) / cores)  
}
```

Partitions *could* be $\text{inputDataSize} / \text{memory per task}$

```
def determinePartitionsFromInputDataSize(inputDataSize: Double) : Int = {  
  Math.round(inputDataSize/availableTaskMemoryMB()).toInt  
}
```

Historical Data

Si possono usare le info storiche del cluster, Stage Metrics, Task Metrics, type of computation, Execution